MongoDB Tutorial: Finding Documents with `find`, `findOne`, Projection, Count, Sort, Limit, and Skip

1. Introduction to Document Retrieval in MongoDB

MongoDB provides two primary methods for retrieving documents from a collection:

1. `find()` – Returns multiple documents matching a query.
2. `findOne()` – Returns only the first document that matches a query.

Both methods allow you to query based on specific fields and values.

---

2. Basic Syntax of `find()` and `findOne()`

2.1 `find()` Method

```
db.<collectionName>.find(<query>)
```

- `<query>` is an optional JSON object defining the field and the value to search for.
- If `<query>` is omitted, all documents in the collection are returned.

Example: Return all documents

```
db.students.find()
```

Example: Return documents where class = "BCA"

```
db.students.find({ class: "BCA" })
```

Example: Return documents where age = 20

```
db.students.find({ age: 20 })
```

Example: Return documents where city = "Delhi"

```
db.students.find({ city: "Delhi" })
```

2.2 `findOne()` Method

```
db.<collectionName>.findOne(<query>)
```

- Works the same as `find`, but returns only the first matching document.

Example:

```
db.students.findOne({ class: "BCA" })
```

- Returns only the first student with class "BCA", even if multiple students qualify.

3. Projection in MongoDB

Projection allows you to control which fields appear in the output.

3.1 Projection using the second parameter in `find()`

```
db.students.find(<query>, { <field>: <1|0> })
```

- `1` → include field
- `0` → exclude field
- `_id` is shown by default unless explicitly excluded.

Examples:

Include only `name` and `age`:

```
db.students.find({ class: "BCA" }, { name: 1, age: 1 })
```

Exclude `_id`:

```
db.students.find({ class: "BCA" }, { name: 1, age: 1, _id: 0 })
```

Include only `name` and `class`:

```
db.students.find({}, { name: 1, class: 1 })
```

3.2 Projection using `.projection()` method

```
db.students
   .find(<query>)
   .projection({ <field>: <1|0> })
```

Example:

```
db.students
   .find({ class: "BCA" })
   .projection({ name: 1, class: 1, _id: 0 })
```

---

4. Additional Query-Enhancing Methods

4.1 Counting Documents – `.count()`

Used to count how many documents match the query.

Example: Count total students

```
db.students.find().count()
```

Example: Count BCA students

```
db.students.find({ class: "BCA" }).count()
```

4.2 Sorting Documents – `.sort()`

```
.sort({ <field>: <1 | -1> })
```

- `1` → Ascending
- `-1` → Descending

Example: Sort by name (A to Z)

```
db.students.find().sort({ name: 1 })
```

Example: Sort by name (Z to A)

```
db.students.find().sort({ name: -1 })
```

Example: Sort by age (ascending)

```
db.students.find().sort({ age: 1 })
```

Example: Sort by age (descending)

```
db.students.find().sort({ age: -1 })
```

With projection:

```
db.students.find({}, { name: 1, age: 1, _id: 0 }).sort({ age: -1 })
```

4.3 Limiting Results – `.limit()`

```
.limit(<number>)
```

Used to restrict the number of returned documents.

Example: Show first 2 students

```
db.students.find().limit(2)
```

4.4 Skipping Results – `.skip()`

```
.skip(<number>)
```

Used for pagination.

Example: Skip first 2 records, show next 2

```
db.students.find().limit(2).skip(2)
```

Example: Skip first 6, show next 3

```
db.students.find().limit(3).skip(6)
```

---

5. Pagination Example with a Query

Example: Fetch BCA students, paginate with limit 2

Page 1:

```
db.students.find({ class: "BCA" }).limit(2).skip(0)
```

Page 2:

```
db.students.find({ class: "BCA" }).limit(2).skip(2)
```

---

6. Summary of All Methods Covered

Core Commands

Purpose	Command
Find multiple documents	`find()`
Find a single document	`findOne()`
Show specific fields	Projection
Count records	`count()`
Sort records	`sort()`
Limit records	`limit()`

Purpose	Command
Skip records	`skip()`

7. Key Takeaways

- `find()` retrieves multiple documents; `findOne()` retrieves only the first one.
- Projection controls visibility of fields.
- Sorting arranges documents by fields in ascending or descending order.
- Limit and skip help in pagination.
- These commands are frequently used in real-world applications, particularly in backend and Node.js-based systems.